# LAPORAN LATIHAN UTS PRAKTIKUM
# PEMROGRAMAN BERBASIS OBJEK




**Oleh :**

**RAYANA JAKA SURYA**

**254107020026**

**TI 2-B / 22**

**LIBRARY**

**PROGRAM STUDI D-IV TEKNIK INFORMATIKA**

**JURUSAN TEKNOLOGI INFORMASI**

**POLITEKNIK NEGERI MALANG**

## Identitas

Nama : Rayana Jaka Surya

NIM : 254107020026

Kelas : TI-2B

Nomor Absen : 22

Link Repository : https://github.com/rayanaaajs/oop-librabry

## Bagian 1: Pengantar

**Penjelasan Konsep & Jawaban "Pikirkan Dulu"**

Dalam pemrograman Java, setiap program butuh titik awal untuk berjalan. Saat kita menekan tombol Run, compiler (Javac) akan menerjemahkan kode yang kita tulis menjadi bytecode. Setelah itu, JVM (Java Virtual Machine) akan menjalankan bytecode tersebut. JVM selalu mencari method bernama main sebagai titik awal eksekusi program. Method main ini harus bersifat static agar JVM bisa langsung menjalankannya tanpa harus membuat objek (instansiasi) dari class tersebut terlebih dahulu.

**Potongan Kode & Penjelasan**

Main.java

```java
public class Main {

    public static void main(String[] args) {
        System.out.println("Welcome to Polinema Library");
    }
}
```

Pada kode di atas, System.out.println digunakan untuk mencetak teks ke layar konsol, diikuti dengan enter (baris baru). Teks yang dicetak harus sama persis dengan spesifikasi agar lolos pengujian autograder.

**Jawaban Uji Pemahaman**

5 contoh benda di perpustakaan sebagai objek, beserta data (atribut) dan aksi (method) yang dimilikinya:

1. Buku: Memiliki data judulBuku dan aksi dipinjam().
2. Rak Buku: Memiliki data kapasitasMaksimal dan aksi tambahBuku().

3. Kartu Anggota: Memiliki data nomorAnggota dan aksi perpanjangMasaAktif().
4. Komputer Pencarian: Memiliki data statusMenyala dan aksi cariKoleksi().
5. Petugas Perpustakaan: Memiliki data namaPetugas dan aksi melayaniPeminjaman().

## Bagian 2: Class dan Object

**Penjelasan Konsep & Jawaban "Pikirkan Dulu"**

Class adalah cetak biru (blueprint), sedangkan Object adalah wujud nyatanya. Pada class Book, data yang perlu diingat adalah isbn, title, year (didapat dari parameter constructor), dan available (diatur sendiri menjadi true saat buku baru dibuat). Agar objek tahu statusnya sedang dipinjam atau tidak, kita menggunakan tipe data boolean pada variabel available. Selain itu, untuk membedakan antara field milik class dan parameter yang namanya sama di dalam constructor, kita menggunakan kata kunci this.

**Potongan Kode & Penjelasan**

Book.java

```
public boolean checkOut() {
        if (!available) {
            return false;
        }
        available = false;
        return true;
    }

public void returnItem() {
        available = true;
    }
```

Method checkOut() mengecek status available. Jika buku tersedia (true), statusnya diubah menjadi false (sedang dipinjam) dan mengembalikan nilai true (berhasil). Jika sudah dipinjam, method langsung mengembalikan false. Method returnItem() bertugas mengembalikan nilai available menjadi true agar buku bisa dipinjam lagi.

**Jawaban Uji Pemahaman**

Output dari kode tersebut adalah:false true Alasannya: Variabel a dan b menunjuk pada objek Book yang sama di dalam memori (heap). Ketika b.checkOut() dipanggil, status objek

tersebut berubah menjadi tidak tersedia. Karena a menunjuk ke objek yang sama, maka a.isAvailable() menghasilkan false. Sementara itu, variabel c membuat objek Book yang baru (berbeda), sehingga nilai isAvailable()-nya masih bawaan dari wujud barunya, yaitu true.

**Kendala dan Solusi**

Kendala: Sempat bingung bagaimana memastikan buku yang sedang dipinjam tidak bisa dipinjam lagi tanpa menggunakan if-else yang berbelit.

Solusi: Membuka Bantuan 1 dan Bantuan 2. Cukup menggunakan pengecekan kondisi boolean sederhana di dalam method checkOut(). Jika statusnya valid untuk dipinjam, ubah field available, lalu return true.

## Bagian 3: Enkapsulasi

**Penjelasan Konsep & Jawaban "Pikirkan Dulu"**

Enkapsulasi adalah konsep menyembunyikan data internal objek dari akses luar sembarangan, biasanya dengan membuat field menjadi private dan menyediakan method public (seperti getter dan setter) untuk mengaksesnya.

Jika loanCount di class Member dibiarkan public, kode dari luar bisa langsung mengubah nilainya sembarangan (misal: member.loanCount = 100;), sehingga merusak aturan maksimal pinjam 3 koleksi. Dengan private, pengubahan nilai hanya bisa lewat method addLoan() yang sudah memiliki pengecekan batas maksimal.

Untuk mencegah penulisan aturan validasi berulang (di constructor dan setter), kita bisa memanggil method setter di dalam constructor. Method canBorrow() tidak memerlukan field tambahan karena statusnya bisa langsung dihitung (dievaluasi) dari nilai loanCount saat itu juga. Terakhir, memberId tidak memiliki setter karena fungsinya sebagai identitas unik yang seharusnya tidak boleh diubah setelah anggota didaftarkan (read-only).

**Potongan Kode & Penjelasan**

Book.java

```java
public class Member {
    private String memberId;
    private String name;
    private int loanCount;

    // Getter untuk akses read-only
    public String getMemberId() {
        return this.memberId;
    }

    // Setter dengan validasi mencegah data kosong
    public void setName(String name) {
        if (name == null || name.trim().isEmpty()) {
            throw new IllegalArgumentException("nama tidak boleh kosong");
        }
        this.name = name;
    }
```

Potongan kode di atas menunjukkan penerapan utuh dari enkapsulasi pada class Member. Semua field dijadikan private. Untuk data memberId yang bersifat read-only, kita hanya menyediakan method getter tanpa setter, sehingga nilainya permanen setelah objek dibuat. Sedangkan untuk data name yang boleh diubah, kita sediakan setter dengan validasi ketat. Pengecekan name == null dilakukan pertama kali untuk menghindari NullPointerException. Jika input tidak valid, program melempar exception dan data lama tetap aman.

**Jawaban Uji Pemahaman**

1. Jika objek Member dibuang, maka objek LibraryCard miliknya akan ikut musnah. Ini adalah ciri relasi Komposisi (kepemilikan kuat), di mana kartu tersebut diciptakan dan eksis hanya untuk anggota tersebut.
2. Jika objek Shelf dibuang, objek Book di dalamnya tidak ikut musnah (masih ada di memori). Ini adalah ciri relasi Agregasi (kepemilikan lemah), di mana rak hanya berfungsi sebagai wadah penampung koleksi yang sebenarnya independen.

**Kendala dan Solusi**

Kendala: Sempat bingung bagaimana memastikan constructor tidak menerima data kosong, karena penulisan aturannya jadi berulang dengan yang ada di setter.

Solusi: Membuka Bantuan 1. Saya menemukan bahwa constructor ternyata bisa memanggil setter miliknya sendiri. Jadi, logika validasinya cukup ditulis satu kali saja di dalam method setter.

## Bagian 4: Relasi Class

**Penjelasan Konsep & Jawaban "Pikirkan Dulu"**

Relasi antar class menunjukkan bagaimana satu objek berhubungan dengan objek lain dalam sistem.

1. Asosiasi (Book & Author): Relasi sejajar/bebas. Objek Book menerima objek Author yang dibuat dari luar (melalui setter). Karena disuplai dari luar, dua buku berbeda boleh menyimpan referensi ke objek Author yang sama persis. Jika objek Book dibuang, objek Author tidak ikut hilang.
2. Komposisi (Member & LibraryCard): Relasi kepemilikan mutlak. Objek Member membuat dan menginisialisasi LibraryCard secara mandiri di dalam constructor-nya, bukan menerima dari parameter luar. Jika Member dibuang, kartunya akan ikut hilang karena kartu tersebut eksis hanya untuk member itu saja.
3. Agregasi (Shelf & Book): Relasi kumpulan (whole-part). Shelf berfungsi sebagai wadah untuk menampung Book menggunakan tipe data array dengan kapasitas 5, namun objek buku ditambahkan dari luar. Arti 0..5 di diagram adalah rak bisa dalam keadaan kosong, atau maksimal menampung 5 buku. Jika rak dihancurkan, data bukunya tetap aman.
4. Dependensi (Librarian): Relasi ketergantungan sesaat. Librarian tidak menyimpan objek sebagai field (tidak mengingat apa pun). Objek Member dan Book hanya digunakan sesaat sebagai parameter di dalam method lend() dan receive().

**Potongan Kode & Penjelasan**

Book.java

```java
// 1. Asosiasi (pada class Book)
private Author author;
public void setAuthor(Author author) {
    this.author = author;
}

// 2. Komposisi (pada class Member)
private LibraryCard card;
public Member(String memberId, String name) {
    this.card = new LibraryCard("CARD-" + this.memberId);
}

// 3. Agregasi (pada class Shelf)
private Book[] books = new Book[5];
private int count = 0;
public boolean addBook(Book book) {
    if (book == null || count >= 5) return false;
    this.books[count++] = book;
    return true;
}

// 4. Dependensi (pada class Librarian)
public boolean lend(Member member, Book book) {
    if (member.canBorrow() && book.isAvailable()) {
        book.checkOut();
        member.addLoan();
        return true;
    }
    return false;
}
```

Asosiasi: Pada Book, field author diisi melalui setAuthor, artinya objek penulis bisa dibagi-bagi (di-share) dengan buku lain.

Komposisi: Pada Member, penggabungan string "CARD-" dan memberId dilakukan langsung saat pembuatan new LibraryCard di constructor, menjamin bahwa setiap kartu terikat kuat dengan member penciptanya.

Agregasi: Pada Shelf, array books dibatasi sampai maksimal 5. Karena ini agregasi, validasi memastikan buku tidak bernilai null dan rak belum penuh sebelum dimasukkan ke array. Dependensi: Pada Librarian, method lend mengecek syarat peminjaman dengan operator && (keduanya wajib true). Jika sukses, status kedua objek diperbarui secara sinkron.

**Jawaban Uji Pemahaman**

Jika objek Member dibuang, maka objek LibraryCard miliknya akan ikut musnah. Ini terjadi karena relasi keduanya adalah Komposisi (kepemilikan kuat), di mana kartu diciptakan oleh dan hidup bergantung pada keberadaan anggotanya.

Jika objek Shelf dibuang, objek Book di dalamnya tidak ikut musnah (masih eksis di memori). Ini terjadi karena relasinya adalah Agregasi (kepemilikan lemah). Rak hanya bertugas sebagai wadah penampung koleksi yang secara logika merupakan objek yang independen.

**Kendala dan Solusi**

Kendala: Pada method pencarian buku findByIsbn di class Shelf, sering terjadi error NullPointerException jika rak belum penuh. memahami konsep relasi class ini menurutku lumayan susah.

Solusi: Membaca Bantuan 1. Saya menyadari bahwa saya harus membatasi perulangan (looping) hanya sampai ke slot yang sudah terisi saja (dengan melacak menggunakan variabel penghitung kapasitas), bukan menelusuri seluruh batas ukuran array yang slot kosongnya masih bernilai null. mempelajari ulang bagian 4 jenis relasi kelas.

## Bagian 5: Inheritance

**Penjelasan Konsep & Jawaban "Pikirkan Dulu"**

Inheritance (pewarisan) adalah konsep di mana sebuah class (disebut subclass atau class anak) mewarisi field dan method dari class lain (disebut superclass atau class induk). Tujuannya agar kita tidak perlu menulis ulang kode yang fungsinya sama. Kalimat "Dvd adalah sebuah LibraryItem" masuk akal karena Dvd merupakan jenis koleksi perpustakaan, sama halnya dengan "Book adalah sebuah LibraryItem" atau "Magazine adalah sebuah LibraryItem". Namun, "Library adalah sebuah Book" jelas tidak masuk akal karena Library (perpustakaan) bukan jenis buku. Field title dan year ditandai dengan # (protected), bukan - (private), agar class turunannya (seperti Book, Dvd, dan Magazine) tetap bisa mengakses field tersebut secara langsung, sementara class di luar hirarki pewarisan tidak bisa. Karena LibraryItem

memiliki constructor yang meminta title dan year, maka saat objek turunan (misalnya Dvd) dibuat, constructor Dvd tersebut wajib mengirimkan data judul dan tahun ke constructor induknya dengan memanggil super(title, year).

**Potongan Kode dan Penjelasan**

Book.java

```
package id.ac.polinema.library;

public class LibraryItem {
    protected String title;
    protected int year;
    private boolean available;

    LibraryItem(String title, int year){
        this.title = title;
        this.year = year;
        this.available = true;
    }
}
```

Proses refactoring dilakukan bertahap. Pertama, logika umum seperti inisialisasi judul, tahun, dan status available dipindahkan dari Book ke LibraryItem. Field umum diubah menjadi protected.

Book.java

```
package id.ac.polinema.library;

public class Dvd extends LibraryItem {
    private int durationMinutes;

    public Dvd(String title, int year, int duratioMinutes){
        super(title, year);
        this.durationMinutes = duratioMinutes;
    }

    public int getDurationMinutes() {
        return durationMinutes;
    }
}
```

Lalu, pada class anak seperti Dvd, kita menggunakan kata kunci extends untuk mewarisi LibraryItem. Baris pertama di dalam constructor anak wajib diisi dengan pemanggilan super() untuk mengeksekusi logika validasi constructor induknya. Sisanya, subclass hanya perlu mengurus data khususnya sendiri (seperti durationMinutes pada Dvd).

**Jawaban Uji Pemahaman**

1. Yang diwarisi Dvd: Field (title, year, available) dan method-method standar (getTitle, checkOut, returnItem, dll) dari LibraryItem.
2. Yang tidak diwarisi Dvd: Constructor milik LibraryItem.
3. Mengapa super(...) tetap harus dipanggil: Meskipun constructor tidak diwarisi, constructor class anak tetap harus memastikan bahwa porsi data milik induk (seperti title dan year) diinisialisasi dan divalidasi dengan benar oleh induknya sendiri, sebelum anak mengurus data khususnya.

**Kendala dan Solusi**

Kendala : sulit untuk memahami kenapa tiap pemanggilan constructor di subclass wajib memakai super() terlebih dahulu sebelum inisialisasi datanya sendiri

Solusi : mencari penjelasan mengenai fungsi super() pada subclass.

## Bagian 6: Overriding dan Overloading

**Penjelasan Konsep & Jawaban "Pikirkan Dulu"**

Overriding adalah saat class anak menulis ulang method dari class induk agar perilakunya sesuai dengan kebutuhan class anak tersebut (memiliki nama dan parameter yang sama persis). Overloading adalah membuat beberapa method atau constructor dengan nama yang sama di dalam satu class, asalkan jumlah atau tipe parameternya berbeda. Compiler akan membedakannya dari parameter yang dikirim saat dipanggil.

Pada method describe() milik Book, penyusunan teks "Judul (Tahun)" sudah dikerjakan oleh parent (LibraryItem), jadi Book cukup memanggil versi parent dengan super.describe() lalu menambahkan teks penulisnya saja di belakang.

Class Magazine tidak perlu memiliki loanDays() sendiri karena aturan standar pinjam 7 hari dari LibraryItem sudah pas untuknya, sehingga versi induk yang akan berjalan.

Untuk menghindari duplikasi kode pada dua constructor Book, constructor 4 parameter bisa memanggil constructor 3 parameter menggunakan kata kunci this(isbn, title, year). Hal yang sama berlaku pada extendLoan() tanpa parameter, yang bisa memanggil extendLoan(int days) dengan nilai bawaan 7 menggunakan this.extendLoan(7).

**Potongan Kode dan Penjelasan**

Book.java

```java
package id.ac.polinema.library;

public class Book extends LibraryItem {
    private String isbn;
    private Author author;

    public Book(String isbn, String title, int year) {
        super(title, year);
        if (isbn == null || isbn.trim().isEmpty()) {
            throw new IllegalArgumentException("isbn tidak boleh kosong");
        }
        this.isbn = isbn;
    }

    public Book(String isbn, String title, int year, Author author) {
        this(isbn, title, year);
        setAuthor(author);
    }

    public String getIsbn() {
        return isbn ;
    }

    public Author getAuthor(){
        return author;
    }

    public void setAuthor(Author author){
        this.author = author;
    }

    @Override
        public int loanDays() {
            return 14;
    }

    @Override
        public String describe(){
            String authorName;
            if (author == null) {
                authorName = "Unknown";
            } else {
                authorName = author.getName();
            }
            return super.describe() + " by " + authorName;
        }
}
```

Pada potongan kode pertama, terlihat konsep Overloading pada constructor Book. Constructor dengan 4 parameter melempar tugas validasi isbn, title, dan year ke constructor 3 parameter menggunakan this(...), sehingga penulisan kode lebih efisien dan tidak berulang.

Pada potongan kode kedua, terlihat konsep Overriding yang ditandai dengan anotasi @Override. Book menimpa method loanDays() agar mengembalikan nilai 14 (bukan 7 seperti induknya). Pada method describe(), class anak memanfaatkan teks yang sudah dihasilkan oleh parent lewat super.describe(), lalu merangkainya dengan nama penulis.

**Jawaban Uji Pemahaman**

Ketika variabel bertipe LibraryItem di-looping dan dipanggil loanDays()-nya, hasilnya bisa berbeda-beda (14, 3, dan 7).Alasannya: Yang menentukan versi method mana yang berjalan

bukanlah tipe variabelnya, melainkan tipe wujud asli objeknya di memori (heap). Ini disebut dengan Dynamic Method Dispatch (Polimorfisme). Jadi, saat variabel menunjuk ke objek Book, maka loanDays() versi Book (14) yang akan dieksekusi secara otomatis saat runtime.

**Kendala dan Solusi**

Kendala: Saat membuat method getTotalLoanDays() di class LibraryItem, saya bingung bagaimana cara mendapatkan nilai lama pinjaman standar yang berbeda-beda untuk Book (14) dan Dvd (3), mengingat posisinya ada di class induk.

Solusi: Membaca Bantuan 2. Ternyata saya tidak boleh menuliskan angka tetap. Saya harus menjumlahkan this.extraDays dengan hasil pemanggilan method this.loanDays(). Karena Polymorphism, this.loanDays() akan otomatis mengambil nilai dari masing-masing subclass sesuai objek yang sedang aktif.

## Bagian 7: Aplikasi Konsol

**Penjelasan Konsep & Jawaban "Pikirkan Dulu"**

Pada pembuatan aplikasi, kita harus memisahkan antara logika bisnis (business logic) dan antarmuka pengguna (user interface). Class Library murni bertugas memproses data, sehingga tidak boleh ada System.out.print di dalamnya. Manfaatnya, jika suatu saat antarmuka aplikasi diubah menjadi GUI (Desktop/Web), class Library sama sekali tidak perlu disentuh karena tidak bergantung pada console.

Array items diizinkan menampung Book, Dvd, dan Magazine sekaligus berkat konsep Polimorfisme (Upcasting). Karena ketiga class tersebut adalah turunan (subclass) dari LibraryItem, maka ketiganya sah diakui sebagai LibraryItem.

LibraryApp menerima Scanner dan PrintStream melalui constructor (konsep Dependency Injection) tidak memakai System.in dan System.out secara langsung. Tujuannya agar aplikasi mudah diuji secara otomatis (automated testing). Autograder bisa dengan mudah mengirimkan input teks simulasi (tanpa keyboard sungguhan) dan menangkap output-nya untuk dinilai.

**Potongan Kode & Penjelasan**

Book.java

```java
// 1. Potongan method di Library.java
public boolean lend(String memberId, String title) {
    Member member = findMember(memberId);
    LibraryItem item = findItemByTitle(title);
    if (member == null || item == null) {
        return false;
    }
    if (member.canBorrow() && item.isAvailable()) {
        item.checkOut();
        member.addLoan();
        return true;
    }
    return false;
}

// 2. Potongan method di LibraryApp.java
private void lendItem() {
    String memberId = readLine("Member ID: ");
    String title = readLine("Title: ");

    if (library.lend(memberId, title)) {
        out.println("Loan successful");
    } else {
        out.println("Loan failed");
    }
}
```

Potongan kode pertama memperlihatkan logika peminjaman di dalam class Library. Method ini mencari keberadaan member dan item terlebih dahulu. Jika keduanya ada, program akan mengecek syarat canBorrow() dan isAvailable(). Jika lolos, status kedua objek diperbarui dan me-return true.

Potongan kode kedua berada di LibraryApp yang bertugas sebagai jembatan dengan pengguna. Method ini murni hanya membaca input via readLine, menyerahkan keputusan pemrosesannya ke library.lend(), lalu mencetak pesan Loan successful atau Loan failed berdasarkan hasil boolean yang dikembalikan.

Book.java

```java
package id.ac.polinema.library;

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        System.out.println("Welcome to Polinema Library");

        Library library = new Library("Polinema Library");
        library.addItem(new Book("978-0132350884", "Clean Code", 2008,
                        new Author("Robert Martin", "USA")));
        library.addItem(new Dvd("Inception", 2010, 148));
        library.addItem(new Magazine("Tempo", 2024, 7));
        library.addMember(new Member("M001", "Budi"));

        Scanner in = new Scanner(System.in);
        LibraryApp app = new LibraryApp(library, in, System.out);
        app.run();
    }
}
```

Pada blok kode di atas, langkah pertama adalah mencetak sapaan awal sesuai instruksi dosen. Kemudian, kita membuat objek Library dan memasukkan beberapa data contoh (1 Buku, 1 DVD, 1 Majalah, dan 1 Anggota) menggunakan method bawaan class tersebut agar aplikasi tidak kosong saat pertama kali dijalankan. Setelah sistem dan data siap, kita membuat perantara UI, yaitu objek LibraryApp, dengan menyertakan System.in dan System.out. Terakhir, method app.run() dipanggil untuk memulai perulangan menu (interaksi dengan user).

**Jawaban Uji Pemahaman (Alur Peminjaman)**

Urutan objek dan method yang berjalan pada satu alur peminjaman dari menu 6 (Lend item) adalah sebagai berikut:

1. Objek LibraryApp: Method lendItem() berjalan. Method ini memanggil readLine() untuk meminta input Member ID dan Title dari pengguna.
2. Objek Library: LibraryApp memanggil library.lend(memberId, title).
3. Objek Library: Di dalam method lend, program memanggil method pembantu findMember(memberId) dan findItemByTitle(title).
4. Objek Member & LibraryItem: Jika objek ditemukan, Library memanggil member.canBorrow() dan item.isAvailable() untuk validasi aturan bisnis.

5. Objek Member & LibraryItem: Jika semua syarat bernilai true, Library mengeksekusi item.checkOut() (untuk mengubah status barang) dan member.addLoan() (untuk menambah jumlah pinjaman anggota).
6. Objek LibraryApp: Kembali ke antarmuka, mencetak output keberhasilan atau kegagalan berdasarkan hasil langkah sebelumnya.

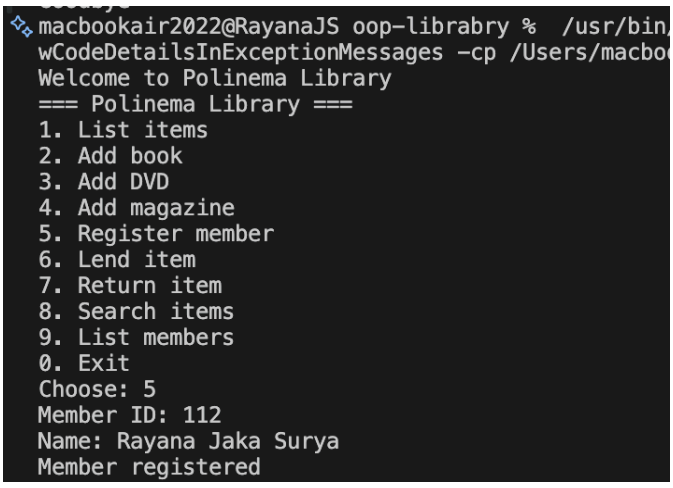
**Kendala dan Solusi**

Kendala: Saat membuat method searchByKeyword di Library, saya bingung bagaimana mengembalikan array hasil yang ukurannya pas (tidak penuh dengan null di belakangnya), padahal jumlah hasilnya belum diketahui sebelum ditelusuri. dan saya merasa kesulitan dan butuh waktu lumayan lama saat menyusun semua class menjadi 1 app yang bisa digunakan oleh user.

Solusi: Membaca Bantuan 1 dan 2. Saya membuat array sementara (temp) dengan ukuran maksimum kapasitas koleksi (20). Setelah proses looping dan pencocokan string (toLowerCase().contains()) selesai, saya menggunakan utilitas bawaan Java yaitu Arrays.copyOf(temp, count) untuk memotong array sementara tersebut menjadi sepanjang count (jumlah hasil yang ditemukan) sebelum me-return nilainya. dan sering berlatih dan belajar konsepnya untuk menyatukan semua class yang udah dibuat diawal sebelum jadi sebuah aplikasi.

## Tambahan Bagian 7

1. **Screenshot Aplikasi Berjalan**




pertama saya melakukan pendaftaran member terlebih dahulu sebelum melakukan peminjaman dengan memilih menu nomor 5 dan mengisi data yang diperlukan.

```
=== Polinema Library ===
1. List items
2. Add book
3. Add DVD
4. Add magazine
5. Register member
6. Lend item
7. Return item
8. Search items
9. List members
0. Exit
Choose: 1
Clean Code (2008) by Robert Martin | Available | 14 days
Inception (2010) [148 min] | Available | 3 days
Tempo (2024) | Available | 7 days
```

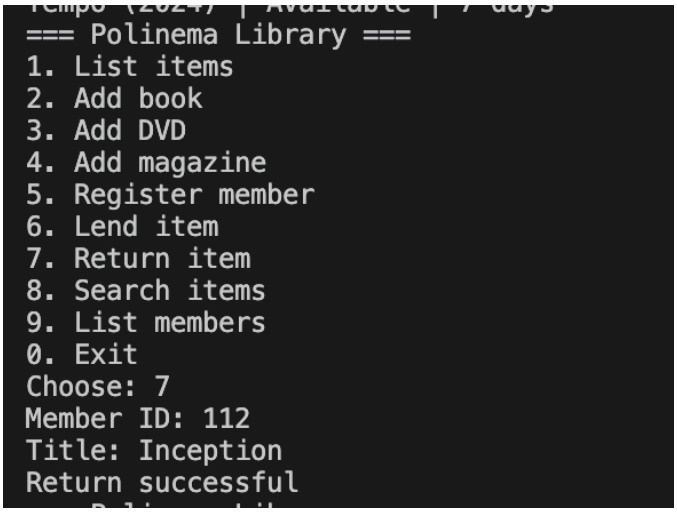
Selanjutnya saya mengecek list barang apa saja yang tersedia diperpustakaan yang bisa saya pinjam dengan memilih menu nomor 1.

```
=== Polinema Library ===
1. List items
2. Add book
3. Add DVD
4. Add magazine
5. Register member
6. Lend item
7. Return item
8. Search items
9. List members
0. Exit
Choose: 6
Member ID: 112
Title: Inception
Loan successful
```

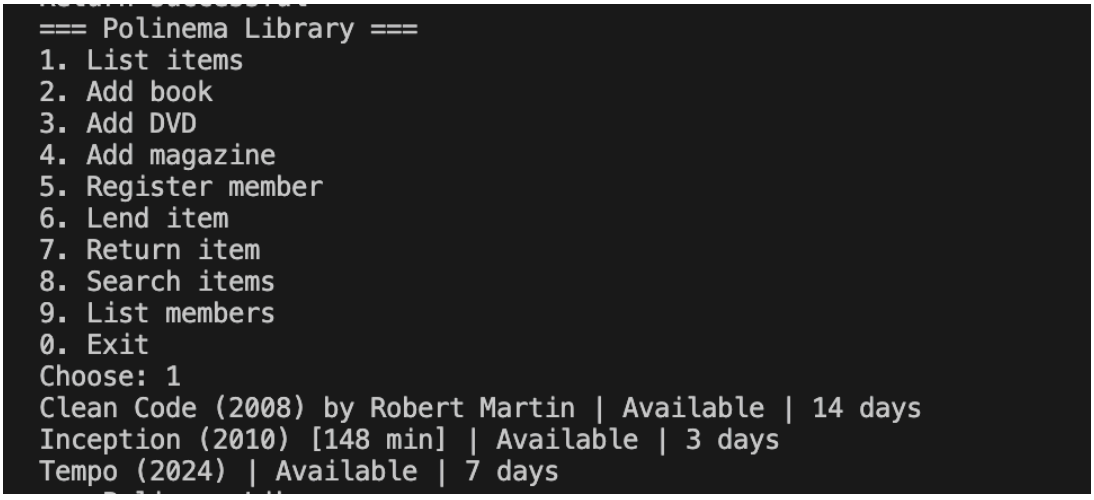
selanjutnya saya melakukan peminjaman item dvd yang berjudul inception dengan memilih menu nomor 6, dan melakukan pengisian data id member dan judul item yang akan dipinjam dan hasilnya sukses karena dvd berjudul inception statusnya tersedia.

```
=== Polinema Library ===
1. List items
2. Add book
3. Add DVD
4. Add magazine
5. Register member
6. Lend item
7. Return item
8. Search items
9. List members
0. Exit
Choose: 1
Clean Code (2008) by Robert Martin | Available | 14 days
Inception (2010) [148 min] | Borrowed | 3 days
Tempo (2024) | Available | 7 days
```

setelah saya meminjam dvd berjudul inception, saya mengecek status item di list item dan hasilnya status dari dvd inception berubah menjadi dipinjam, dan orang lain tidak bisa meminjam item tersebut.




selanjutnya saya mengembalikan item dvd yang berjudul inception yang saya pinjam dengan memilih menu nomor 7, dan melakukan pengisian data id member dan judul item yang sudah saya pinjam dan hasilnya sukses karena dvd berjudul inception memang dipinjam oleh orang dengan member id 112.




dan yang terakhir saya mengecek list item lagi dan hasilnya dvd dengan judul inception statusnya sudah tersedia karena barangnya sudah dikembalikan oleh orang yang meminjam.

2. **Penjelasan Kerja Sama LibraryApp, Library, Member, dan LibraryItem pada Satu Alur Peminjaman**

   Pada alur peminjaman (menu 6 "Lend item"), keempat class ini bekerja sama dengan peran yang memisahkan antara antarmuka (UI) dan logika dari aplikasi tersebut:

   1. LibraryApp (Sebagai Antarmuka / UI):

      Proses dimulai dari method lendItem() di LibraryApp. Class ini bertugas berinteraksi dengan pengguna, yaitu meminta dan membaca input Member ID dan Title melalui keyboard (menggunakan fungsi readLine()). Setelah mendapat input, LibraryApp memanggil method library.lend(memberId, title) dan menunggu hasil kembaliannya (boolean). LibraryApp tidak melakukan pengecekan aturan perpustakaan sama sekali.

   2. Library (Sebagai Koordinator / Controller):

      Begitu method lend() dipanggil, Library mengambil alih kendali. Pertama, ia mencari wujud objek anggota dan koleksi di dalam array menggunakan method findMember() dan findItemByTitle(). Jika objeknya ditemukan, Library tidak mengubah nilainya secara langsung, melainkan bertanya terlebih dahulu kepada objek Member dan LibraryItem terkait apakah transaksi ini diizinkan.

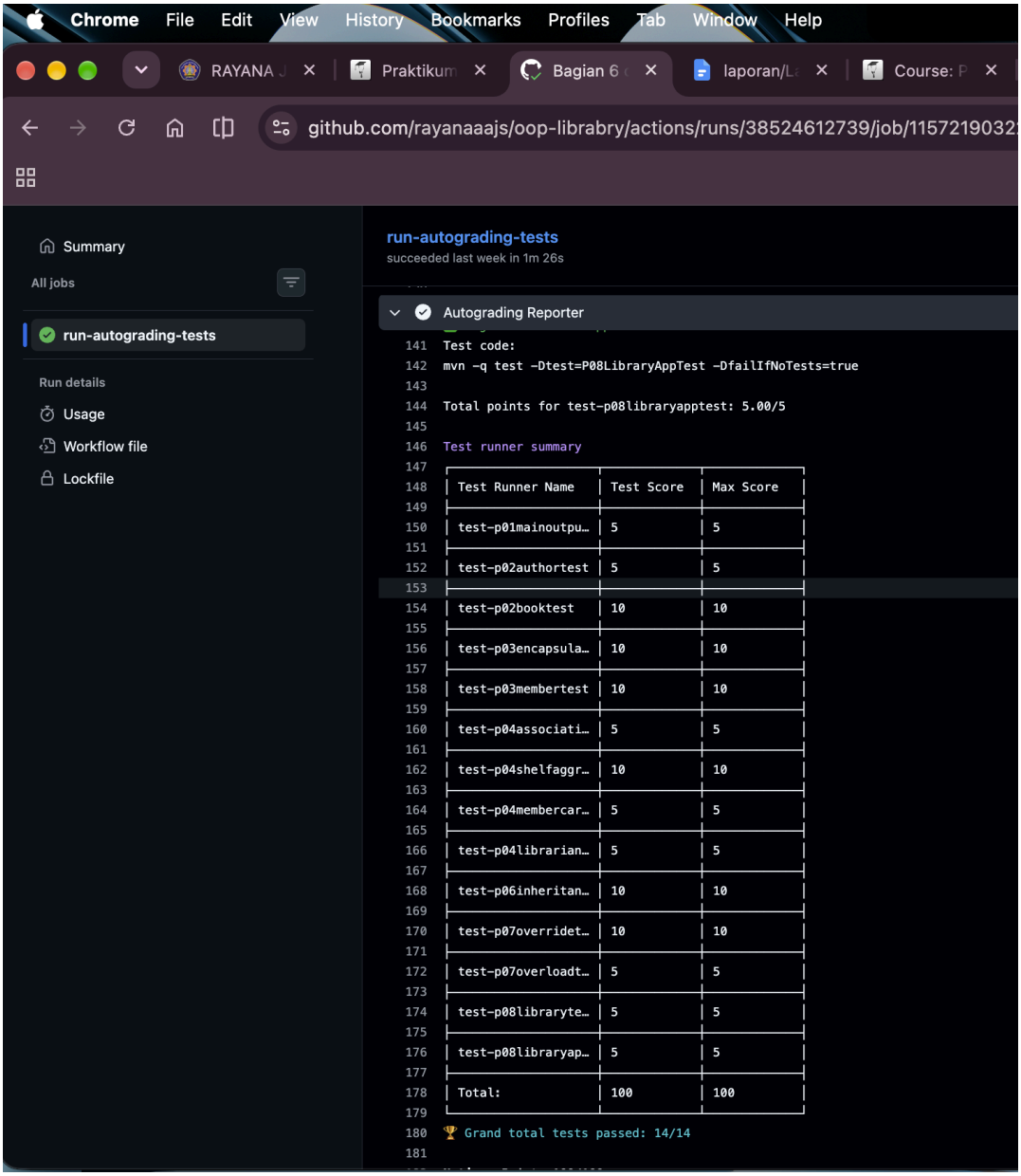
   3. Member (Sebagai Pemilik Data Anggota):

      Library memanggil method member.canBorrow(). Class Member akan mengevaluasi dirinya sendiri, mengecek apakah nilai loanCount miliknya masih di bawah 3. Jika ya, ia me-return true. Jika peminjaman disetujui secara keseluruhan, Library akan memanggil member.addLoan() agar class Member menambahkan jumlah pinjamannya sendiri.

   4. LibraryItem (Sebagai Pemilik Data Koleksi):

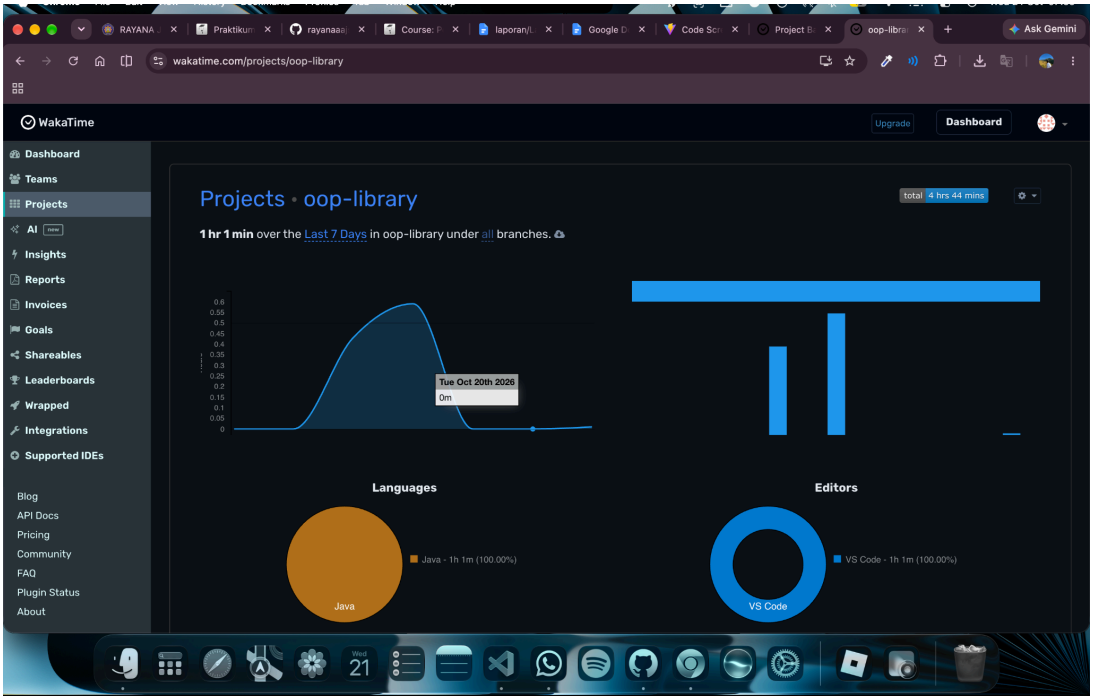
      Bersamaan dengan pengecekan anggota, Library juga memanggil method item.isAvailable(). Class LibraryItem (atau subclass-nya seperti Book, Dvd, Magazine) akan mengecek field status miliknya sendiri. Jika buku sedang tidak dipinjam orang lain, ia me-return true. Saat peminjaman disetujui, Library memanggil item.checkOut() agar koleksi tersebut memperbarui statusnya sendiri menjadi sedang dipinjam (available = false).

## Hasil Pengujian :




dari hasil pengujian saya sudah menyelesaikan semua bagian dan sudah sesuai ketentuan yang dites oleh autograder dengan total point 100, dengan 7 kali commit per bagian, dan 7 kali push ke repository github

## Bukti Wakatime :




screenshot diatas yaitu proses pengerjaan latihan uts ini yang direkam oleh wakatime, dengan total waktu 4 jam 44 menit yang aku kerjakan yaitu pada hari selasa, 13 oktober 2026.

## Refleksi

Secara keseluruhan, konsep yang paling lumayan menantang dan memakan banyak waktu bagi saya adalah pada saat mengerjakan Bagian 7 (Aplikasi Konsol). Di bagian ini, tantangan terbesarnya bukan lagi soal memahami sintaks dasar Java, melainkan bagaimana merangkai seluruh class yang sudah dibangun pada bagian-bagian sebelumnya (Book, Member, LibraryItem, dll) agar bisa saling berkomunikasi dan bekerja sama secara terstruktur di dalam class Library dan LibraryApp.

Memisahkan antara logika bisnis (di dalam class Library) dengan antarmuka pengguna (di dalam class LibraryApp) ternyata cukup membingungkan pada awalnya. Saya harus benar-benar memastikan bahwa Library murni hanya memproses data tanpa mencetak apapun, sedangkan LibraryApp hanya bertugas membaca input dan menampilkan hasil berdasarkan kembalian (return value) dari Library. Selain itu, menerapkan polimorfisme saat memasukkan berbagai subclass (Book, Dvd, Magazine) ke dalam satu array bertipe

LibraryItem[] memberikan pemahaman baru bagi saya tentang seberapa fleksibel dan rapinya penulisan kode jika kita menggunakan Inheritance dengan benar.

Rencana Belajar ke Depannya, saya berencana untuk:

1. Memperdalam pemahaman tentang struktur data selain Array statis (seperti ArrayList atau Collection Framework lainnya), karena memanipulasi array biasa seperti memotong ukurannya dengan Arrays.copyOf() cukup rawan terjadi error NullPointerException.
2. Lebih banyak berlatih merancang struktur class (seperti membaca UML Class Diagram) sebelum menulis kode, karena saya menyadari bahwa perencanaan blueprint yang matang sangat menentukan kelancaran saat membangun aplikasi secara utuh (seperti pada Bagian 7).
3. Mempelajari lebih lanjut tentang penanganan exception tingkat lanjut agar aplikasi tidak mudah crash jika menerima input pengguna yang benar-benar di luar ekspektasi.